Projet : Cloud-Native Trade Anomaly Detection & Reporting Platform (DNEXT) (Project 3)

Groupe : 11

Etudiant : Adnan Binjos, Théo Falcinelli, Alexandre Schmid

Lien Git : https://github.com/AdnanBinBin/dnext-group11-project3.git

Lien vidéo: https://streamable.com/ajj2q8

Architecture

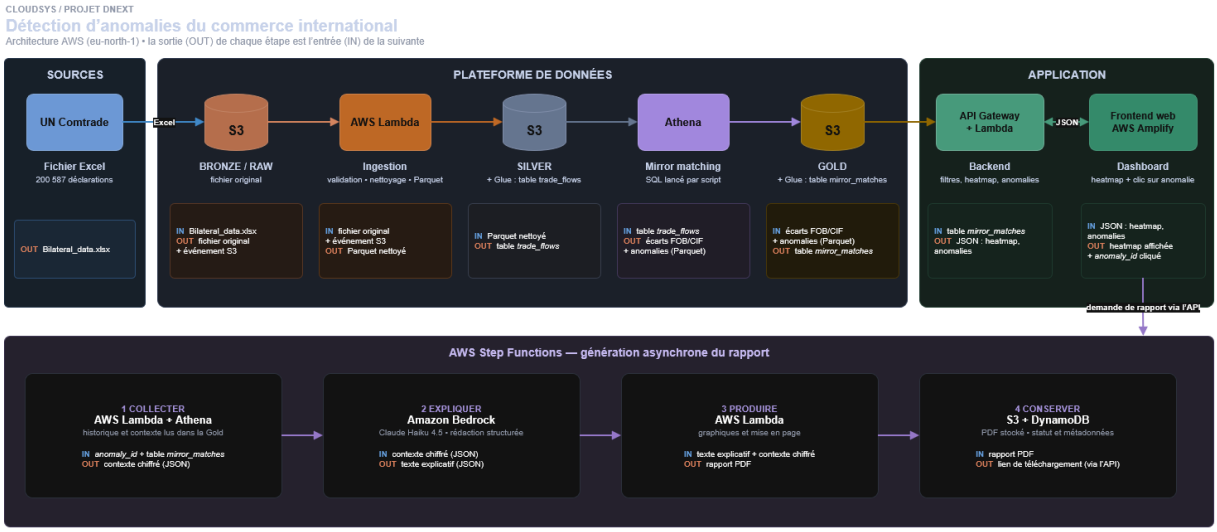


Figure 1 : Architecture AWS de la plateforme (région eu-north-1)

1. **Composants** : chaque composant de la figure est expliqué ici.
    - **UN Comtrade (source)** : fichier Excel de 200 587 déclarations mensuelles d'import et d'export (10 pays exportateurs, 7 produits agricoles, depuis 2019).
    - **S3 Bronze / Raw** : conserve le fichier original dans un bucket versionné ; son dépôt déclenche automatiquement l'ingestion.
    - **Lambda d'ingestion** : valide et normalise chaque ligne, met de côté les lignes invalides avec leur motif et écrit les données au format Parquet.
    - **S3 Silver + Glue (trade_flows)** : données nettoyées en Parquet, partitionnées par année et décrites dans le Glue Data Catalog pour être interrogées en SQL.
    - **Athena (mirror matching)** : rapproche en SQL les déclarations d'export (FOB) et d'import (CIF), calcule les prix unitaires, les écarts et les anomalies.
    - **S3 Gold + Glue (mirror_matches)** : résultats précalculés (142 451 flux, dont 27 467 anomalies), consultés sans relancer le calcul.
    - **API Gateway + Lambda (backend)** : API HTTP qui interroge la Gold avec Athena et renvoie en JSON les filtres, la heatmap et les anomalies.
    - **Frontend web (AWS Amplify)** : application Vue.js affichant la heatmap partenaires × périodes ; un clic sur une anomalie permet de demander un rapport.
    - **AWS Step Functions** : orchestre de manière asynchrone la génération d'un rapport en quatre étapes, avec nouvelles tentatives en cas d'erreur.
    - **1. Collecter (Lambda + Athena)** : rassemble le contexte chiffré de l'anomalie : historique sur ±12 mois et prix vers les autres partenaires.
    - **2. Expliquer (Amazon Bedrock)** : le modèle Claude Haiku 4.5, appelé par une Lambda, rédige une explication structurée à partir de ce contexte.
    - **3. Produire (Lambda)** : génère les graphiques et met en page le rapport PDF à partir des chiffres et du texte.
    - **4. Conserver (S3 + DynamoDB)** : le PDF est stocké dans S3 ; DynamoDB conserve l'état de la demande et permet de réutiliser un rapport existant.

- **Infrastructure as a code** : CloudFormation pour le déploiement de l'ensemble par scripts.

Déploiement

1. **Stockage** : les données (Bronze, Silver, Gold) et les rapports PDF sont stockés sur Amazon S3, dans des buckets.
2. **Catalogue et requêtes** : les schémas sont référencés dans AWS Glue Data Catalog ; le mirror matching et les consultations sont exécutés en SQL par Amazon Athena, sans serveur
3. **Application** : elle est représentée par un ensemble de FaaS (AWS Lambda, codé en Python) : ingestion (déclenchée par un événement S3), backend de consultation, et api, collect, narrate et render pour les rapports.
4. **Exposition** : les routes HTTP sont exposées par Amazon API Gateway (HTTP API), partagée par le backend et les rapports ; aucun load balancer n'est nécessaire, tout est géré par AWS.
5. **Orchestration** : la génération des rapports est un workflow AWS Step Functions (type Standard).
6. **Modèle d'IA** : Claude Haiku 4.5 (Anthropic), utilisé via Amazon Bedrock avec un profil d'inférence européen.
7. **État des rapports** : les demandes et leur statut sont conservés dans Amazon DynamoDB, facturé à la requête.
8. **Interface web** : application Vue.js 3, construite avec Vite sous Node.js 22, hébergée sur AWS Amplify Hosting.
9. **Infrastructure** : l'ensemble est décrit en AWS CloudFormation (cinq stacks) et déployé par des scripts Bash ; aucune VM ni aucun cluster Kubernetes n'est utilisé.

Tâches

Tâches	Résultats	Personne impliquée
Conception de l'architecture	Architecture AWS avec les entrées et sorties de chaque composant (Figure 1)	L'équipe
Étude comparative des services cloud	Alternatives comparées pour chaque bloc ; retenus : S3 + Glue + Athena, Lambda, Amplify, Bedrock et Step Functions	L'équipe
Mise en place DB Bronze	Mise en place du bucket S3 qui fait une copie des données du système source	Théo Falcinelli
Cleaning des données (Bronze → Silver)	200 587 déclarations validées et écrites en Parquet, aucune ligne rejetée, table Glue trade_flows	Théo Falcinelli
Mirror matching et couche Gold	Rapprochement SQL avec Athena : 142 451 flux, dont 27 467 anomalies (table mirror_matches)	Alexandre Schmid
API backend	Une Lambda et cinq routes HTTP documentées en OpenAPI, avec limites de débit	Alexandre Schmid
Déploiement et tests	Cinq stacks CloudFormation, scripts de test, de déploiement et de suppression, tests unitaires sans accès AWS	L'équipe
Frontend	Application Vue.js sur Amplify : heatmap, détail des anomalies et demande de rapport	Adnan Binjos
Génération des rapports IA	Workflow Step Functions et Bedrock : rapport PDF en 30 à 80 s	Adnan Binjos
Scalabilité, coûts	Limites du MVP et évolutions proposées : tests de charge	Adnan Binjos